

E065

JavaScript Programming

Armored Drill

8 concepts · 12 items · 4 transfer items

What this is. Every concept in this course's public description, re-asked with more disguise than a practice test uses. Each stem carries facts that do not decide it. Every wrong answer is a named trap.

Source. Scope from the course's public description in the WGU Institutional Catalog (September 2026); items written from the field's standard body of knowledge. No WGU assessment item, course material or proprietary content is used or reproduced. Not affiliated with or endorsed by Western Governors University.

Method © 2026 Erik Daniel Lindquist — Course Mastery Coach, armored drill. Items drafted with AI assistance under his direction.

How to run it

Paper first. Pencil. One pass, no notes, a timer if you want the pressure.

1. **Before you read the options**, fill the strip under the item: the target word (what is actually being asked), and the one fact in the stem that decides it.
2. **Principle before arithmetic**. Name the rule, then compute.
3. **Camouflage**. Most of every stem does not matter. Ask: which single fact decides this?
4. **Inverse pairs are the bait**. If two options are opposites, one of them is the trap built for you.
5. **Mark sure, unsure or guess** honestly. Sure-and-wrong is the most valuable miss on the page — it is a wrong mental model.
6. **Absolutes** (always, never, guarantee) are usually wrong. And recognizing an answer is not the same as owning it.
7. Items with a **heavy black bar** on the left are **transfer items**: the same principle, reversed or solved for a different variable.
8. Stop at the STOP page. Then grade, fill the triage grid, and tally the traps you fell for. The biggest tally is what you drill next.

The strip, filled in, looks like this:

Target word _____	The one fact that decides _____	Answer _____	[] sure [] unsure [] guess
-----------------------------	---	------------------------	----------------------------------

Items

1.

On a recipe site that gets 8,000 visits a day, a developer writes `const settings = { theme: 'light' }`; and later runs `settings.theme = 'dark'`. What happens?

- A. The theme property changes to 'dark' with no error.
- B. A `TypeError` is thrown because `settings` is `const`.
- C. It fails only on high-traffic pages.
- D. The change is silently ignored.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

2. [TRANSFER — reversed stem / different variable]

Which line throws a `TypeError`?

- A. `let total = 0; total = 5;`
- B. `const items = [1, 2, 3]; items.push(4);`
- C. `const count = 0; count = count + 1;`
- D. `var x = 1; var x = 2;`

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

3.

A bakery's order form, styled with a new brand color palette, has a quantity box. The code reads `const qty = input.value;` and checks if `(qty === 5)`. A customer types 5, but the check is false. What is the fix?

- A. Convert first: `Number(input.value) === 5`.
- B. The new color palette is blocking the input value.
- C. Always use `==` so types never matter.
- D. Compare with `qty === '5.0'`.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

4. [TRANSFER — reversed stem / different variable]

Which expression evaluates to true?

- A. `null === undefined`
- B. `NaN === NaN`
- C. `0 == '0'`
- D. `0 === '0'`

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

5.

A portfolio page shows 12 project cards with class card. The developer runs `document.querySelector('.card').classList.add('highlight')`. The CSS file loads from a CDN in under 50 ms. Only the first card is highlighted. What fixes it?

- A. Set `innerHTML` on the first card.
- B. Use `querySelectorAll('.card')` and loop with `forEach` to add the class.
- C. Use `getElementById('card')`.
- D. Load the CSS file before the script.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

6.

A sign-up form with six fields and a custom web font clears itself and reloads the page when the user clicks Submit, so the script never shows its thank-you message. What should the submit handler do?

- A. Call `event.preventDefault()` in the form's submit listener.
- B. Call `event.stopPropagation()`.
- C. Remove the custom font so the page loads faster.
- D. Move the listener to the button's click event only.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

7. [TRANSFER — reversed stem / different variable]

Clicking a Delete button inside a card also fires the card's own click listener, which opens the card. How do you keep the card's listener from firing for that click?

- A. Add the button listener in the capture phase.
- B. Call `event.preventDefault()` in the button's handler.
- C. Call `event.stopPropagation()` in the button's handler.
- D. Remove the card's click listener entirely.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

8.

A to-do app with a pastel color scheme chosen by a focus group attaches a click listener to every li at page load. Tasks added later with appendChild do not respond to clicks. What is the best fix?

- A. Call preventDefault on the list.
- B. Put one listener on the ul and use event.target.closest('li') to find the task.
- C. Run querySelectorAll('li') again at page load.
- D. Attach the listeners inside the window load event.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

9.

A shop saves a cart array of three product objects with localStorage.setItem('cart', cart). The shop ships only to 14 states. When the page reloads, localStorage.getItem('cart') returns the text [object Object],[object Object],[object Object]. What fixes it?

- A. Use sessionStorage instead.
- B. Call JSON.parse when saving and JSON.stringify when reading.
- C. Limit the cart to one product.
- D. Save with JSON.stringify(cart) and read with JSON.parse.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

10. [TRANSFER — reversed stem / different variable]

A user closes the tab and returns to the site the next day. Where must a saved preference be stored so it is still there?

- A. A JavaScript variable
- B. A CSS custom property
- C. sessionStorage
- D. localStorage

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

11.

A community forum that uses HTTPS displays each user's comment under a post. A developer plans to write `div.innerHTML = comment`. What is the safer choice?

- A.** Use `innerHTML` but add the comment twice to check it.
- B.** Keep `innerHTML`, since HTTPS already protects the page.
- C.** Never display user comments at all.
- D.** Set `div.textContent = comment`.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

12.

A weather widget runs `let data; fetch(url).then(r => r.json()).then(j => { data = j; }); console.log(data);`. The API usually responds in about 200 ms. The console shows `undefined`. Why?

- A.** `fetch` returns the data directly, so read `r.data` instead.
- B.** `console.log` runs before the promise resolves; use `await` or `log` inside `.then`.
- C.** Wrap `console.log` in `setTimeout(..., 0)`.
- D.** The API must be down.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

STOP

Answers and dissection follow. Finish every item and every strip first.

Key and dissection

Struck-through text is the camouflage: facts that do not decide the item.

1. Answer A Concept 1

On a recipe site ~~that gets 8,000 visits a day~~, a developer writes `const settings = { theme: 'light' }`; and later runs `settings.theme = 'dark'`. What happens?

Target word: effect of changing a property on a const object **Decides:** const blocks rebinding, not changing the object's contents.

A	Correct	The binding still points to the same object; only its property changed.
B	Confusion set	That happens on reassignment (<code>settings = ...</code>), not on property change.
C	Camouflage	Traffic has nothing to do with language rules.
D	Adjacent category	Describes a frozen object (<code>Object.freeze</code>), not const.

2. Answer C Concept 1 · transfer

Which line throws a `TypeError`?

Target word: find the illegal reassignment **Decides:** Reassigning a const binding is the error.

A	Adjacent category	<code>let</code> allows reassignment.
B	Inverted operation	Mutates the array but does not rebind; this is allowed.
C	Correct	Reassigns a const binding.
D	Confusion set	<code>var</code> allows redeclaration without error.

3. Answer A Concept 2

A bakery's order form, ~~styled with a new brand color palette~~, has a quantity box. The code reads `const qty = input.value`; and checks if (`qty === 5`). A customer types 5, but the check is false. What is the fix?

Target word: why the strict check fails **Decides:** `input.value` is the string '5'; `===` does not convert types.

A	Correct	Now both sides are numbers.
B	Camouflage	CSS does not change the value's type.
C	Absolute language	Loose equality hides type bugs; convert deliberately instead.
D	Dropped element	Still a string compare, and '5' is not '5.0'.

4. Answer C Concept 2 · transfer

Which expression evaluates to true?

Target word: which comparison is true **Decides:** `==` converts '0' to 0 before comparing.

A	Confusion set	They are loosely equal but different types: false.
B	Adjacent category	<code>NaN</code> is never equal to itself.
C	Correct	Loose equality converts the string to a number.
D	Inverted operation	Strict equality sees number vs. string: false.

5. Answer B Concept 3

A portfolio page shows 12 project cards with class card. The developer runs `document.querySelector('.card').classList.add('highlight')`. ~~The CSS file loads from a CDN in under 50 ms.~~ Only the first card is highlighted. What fixes it?

Target word: select all matches **Decides:** `querySelector` returns only the first match.

A	Adjacent category	Changes content, not which elements are selected.
B	Correct	Returns every match.
C	Confusion set	Finds one element by id, and these cards use a class.
D	Camouflage	Load order does not change what <code>querySelector</code> returns.

6. Answer A Concept 4

A sign-up form ~~with six fields and a custom web font~~ clears itself and reloads the page when the user clicks Submit, so the script never shows its thank-you message. What should the submit handler do?

Target word: stop the default submit **Decides:** Submitting a form reloads the page by default.

A	Correct	Stops the browser's default submit and reload.
B	Confusion set	Stops bubbling to parents, not the default action.
C	Camouflage	The font has nothing to do with the reload.
D	Adjacent category	The form still submits by default; pressing Enter also bypasses it.

7. Answer C Concept 4 · transfer

Clicking a Delete button inside a card also fires the card's own click listener, which opens the card. How do you keep the card's listener from firing for that click?

Target word: stop bubbling, not the default **Decides:** The click bubbles from the button up to the card.

A	Adjacent category	Changes when the button runs, not whether the card also runs.
B	Inverted operation	Blocks default actions; the event still bubbles.
C	Correct	Stops the event from reaching the card's listener.
D	Absolute language	Breaks opening the card for every other click.

8. Answer B Concept 5

A to-do app ~~with a pastel color scheme chosen by a focus group~~ attaches a click listener to every li at page load. Tasks added later with `appendChild` do not respond to clicks. What is the best fix?

Target word: handle clicks on dynamic elements **Decides:** New elements have no listener; a parent does.

A	Adjacent category	Controls default actions, not whether a listener exists.
B	Correct	Event delegation catches clicks from current and future children.
C	Dropped element	Still runs only once, before new items exist.
D	Confusion set	Still before tasks are added later.

9. Answer D Concept 6

A shop saves a cart array of three product objects with `localStorage.setItem('cart', cart)`. ~~The shop ships only to 14 states.~~ When the page reloads, `localStorage.getItem('cart')` returns the text `[object Object],[object Object],[object Object]`. What fixes it?

Target word: store objects correctly **Decides:** Web Storage stores strings only.

A	Adjacent category	Also stores only strings.
B	Inverted operation	The two steps are reversed.
C	Dropped element	One object still turns into <code>[object Object]</code> .
D	Correct	Turns objects into a JSON string and back.

10. Answer D Concept 6 · transfer

A user closes the tab and returns to the site the next day. Where must a saved preference be stored so it is still there?

Target word: which store persists **Decides:** Only `localStorage` survives closing the tab.

A	Dropped element	Lost on any reload.
B	Adjacent category	Styling, not storage.
C	Confusion set	Cleared when the tab closes.
D	Correct	Persists until cleared.

11. Answer D Concept 7

A community forum ~~that uses HTTPS~~ displays each user's comment under a post. A developer plans to write `div.innerHTML = comment`. What is the safer choice?

Target word: safe way to show untrusted text **Decides:** The comment is user-supplied and could contain markup.

A	Dropped element	Still parses untrusted HTML.
B	Camouflage	HTTPS protects data in transit, not what the page renders.
C	Absolute language	Safe display is possible with <code>textContent</code> .
D	Correct	Shows the text literally; any markup is not run.

12. Answer B Concept 8

A weather widget runs `let data; fetch(url).then(r => r.json()).then(j => { data = j; }); console.log(data);`. ~~The API usually responds in about 200 ms.~~ The console shows `undefined`. Why?

Target word: why the value is missing **Decides:** `fetch` is asynchronous; `console.log` runs first.

A	Confusion set	<code>fetch</code> returns a promise of a <code>Response</code> , not the data.
B	Correct	The response arrives after the next line has already run.
C	Adjacent category	Defers only to the next task, not until the network reply.
D	Camouflage	The stem says it responds; the timing is the issue.

Triage grid

Write each item number in the box it belongs to.

	Right	Wrong
Sure	Owned.	Concept inversion — top priority. A wrong mental model; it survives rewording.
Unsure / guess	Recognition, not mastery. Re-drill with the transfer.	Not learned yet.

Trap tally

For every miss, find the letter you chose in the key and tally its trap. The biggest number is the pattern to drill.

Trap	What it does	In this drill	Your misses
Dropped element	Leaves out a required term or step.	5	
Wrong stem number	A figure copied from the prompt that is not the answer.	0	
Adjacent category	The right idea in the wrong bucket.	10	
Inverted operation	Direction flipped: add/subtract, cause/effect, higher/lower.	4	
Absolute language	Always, never, guarantee — where judgment applies.	3	
Confusion set	Every option is real; the test is exact role-matching.	8	
Camouflage	Answers a stem fact that was never the question.	6	

Anchor rows — the concept list

One row per concept: the rule to lock in, the trap that catches people, and the tell in the question that signals it.

#	Concept	Rule	Trap	Tell
1	const blocks reassignment, not mutation	A const binding cannot be pointed at a new value, but the object or array it holds can still be changed.	Believing const makes an object read-only.	A const object or array followed by a property change or push.
2	Strict vs. loose equality	=== compares value and type with no conversion; == converts types first. Form input values are always strings.	Comparing an input's value to a number with === and expecting a match.	A value read from a form field compared to a number.
3	querySelector vs. querySelect orAll	querySelector returns only the first matching element; querySelectorAll returns a NodeList of every match to loop over.	Expecting querySelector to change every element with a class.	Many matching elements, only one changes.
4	preventDefault vs. stopPropagation	preventDefault stops the browser's default action (form submit, link follow); stopPropagation stops the event from bubbling to parent listeners.	Using one when the problem needs the other.	Page reloads on submit (default) vs. a parent handler also fires (bubbling).
5	Event delegation	Attach one listener to a stable parent and use event.target to handle children, including ones added later.	Attaching listeners to items at load and expecting new items to respond.	Items added dynamically do not react to clicks.
6	Web Storage holds strings	localStorage and sessionStorage store strings only; JSON.stringify to save objects and JSON.parse to read them. localStorage persists; sessionStorage ends with the tab.	Saving an array of objects directly, or using sessionStorage for data that must survive closing the tab.	Data comes back as [object Object], or is gone the next day.
7	textContent vs. innerHTML for user input	Insert untrusted text with textContent so it is shown as text, not parsed as HTML.	Using innerHTML for user comments because it is convenient.	User-supplied text being written into the page.
8	Asynchronous fetch and promises	fetch returns a promise; code after it runs before the response arrives, so use await or put the work inside .then.	Reading the result on the next line and expecting it to be filled in.	A variable is undefined right after a fetch call.

Course scope (public description): JavaScript Programming expands students' web development skill set by teaching them to build interactive, user- centered websites using JavaScript, cascading style sheets (CSS), the Document Object Model (DOM), and event handling. The course also introduces dynamic navigation, reactive interface components, and client-side data entry and storage. Web Development Foundations is a prerequisite for this course.